ITM SKILLS UNIVERSITY

School of Future Tech

Case Study Number 85 – Report

on

Smart Toll Collection System (C++)

by

Subrata Panda

150096725110

Cohort: Larry Page

Academic Year 2025–2029

Index

No.	Section	Page
1	Introduction to the Case Study	3
2	Problem Statement / Case Background (Abstract)	4
3	Case Study Design	5
4	Methods & Technology Applied	6
5	Implementation Details	7
6	Results and Conclusion	8
7	References	9

1. Introduction to the Case Study

Highway toll collection is an essential process for maintaining road infrastructure and managing transportation revenue. Traditionally, toll collection has been performed manually where vehicles stop at toll booths and make payments to operators. This manual process often results in traffic congestion, delays, and difficulties in maintaining accurate transaction records.

To improve efficiency, toll management systems can be automated using software that handles vehicle identification, toll calculation, and transaction recording. Automation reduces manual work, speeds up toll processing, and ensures better data management.

This project presents the design and implementation of a Smart Toll Collection System using C++. The system simulates the operations of a toll plaza by allowing vehicles to be registered, processing toll payments, and storing transaction records. The application uses object-oriented programming principles and file handling techniques to organise data and manage toll transactions efficiently.

Key Features at a Glance

- Vehicle registration and identification using plate numbers.
- Automatic toll calculation based on vehicle type.
- Transaction logging using file handling.
- Receipt generation for toll payments.
- Menu-driven console interface for user interaction.

2. Problem Statement / Case Background (Abstract)

Background

Manualtoll collection systems require vehicles to stop at toll booths and make payments directly to toll operators. This process can lead to several operational challenges such as long waiting times, traffic congestion, and difficulties in maintaining proper records.

Some common problems associated with manual toll collection include:

- Traffic congestion due to vehicles stopping at toll booths.
- Manual errors during toll calculation or record keeping.
- Lack of structured transaction records.
- Slow vehicle processing during peak traffic periods.

A computerised toll management system can address these issues by automating toll calculation and storing transaction data digitally.

Abstract

Thiscasestudy presents the development of a Smart Toll Collection System implemented using C++. The system allows users to register vehicles, process toll payments, and maintain transaction records through a menu-driven interface.

The application uses object-oriented programming concepts to represent vehicles, toll booths, and system operations. Toll charges are calculated automatically based on the type of vehicle. All toll transactions are recorded in a text file, allowing the system to maintain a history of payments. The system also generates receipts for each toll transaction .
This project demonstrates how programming techniques such as classes, arrays, functions, and file handling can be applied to simulate a real-world toll management system.

3. Case Study Design

The Smart Toll Collection System is designed using a modular object-oriented architecture. The system is divided into multiple components, each responsible for specific functionality such as vehicle management, toll calculation, and transaction logging.

System Architecture

Layer	Component / Description
User Interface Layer	Console-based menu interface
Application Logic Layer	C++ classes controlling toll operations
Data Management Layer	Vehicle records stored in arrays
Storage Layer	Transaction and receipt files

Main System Components

Vehicle Class The Vehicle class stores information related to vehicles including plate number, owner name, and vehicle
type. This class is responsible for storing and displaying vehicle information.

TollBooth Class

The TollBooth class represents individual toll booths and stores details such as booth ID, booth location, total revenue collected, and number of vehicles processed. Each booth keeps track of its operational statistics.

TollSystem Class

The TollSystem class controls the overall functionality of the program. Its responsibilities include registering vehicles, processing toll payments, calculating toll charges, logging transaction records, generating receipts.

Application Flow

1. The user launches the program and views the main menu.
2. A vehicle can be registered by entering its details.
3. The system calculates the toll based on the vehicle type.
4. The toll payment is processed and recorded in a transaction log.
5. A receipt is generated for the payment.
6. The system updates booth statistics including total revenue and vehicle count.

4. Methods & Technology Applied

4.1 Technology Stack

Category	Technology
Programming Language	C++
Programming Paradigm	Object-Oriented Programming
Data	Arrays
Storage File Handling	Text Files
Interface	Console-based UI

4.2 Object-Oriented Programming

Thesystem usesclasses andobjects to represent real-world entities such as vehicles and toll booths. This improves program organisation and makes the code easier to maintain and expand. Encapsulation ensures that each class manages its own data, while the TollSystem class coordinates interactions between components.

4.3 Toll Calculation Logic

The toll amount isdetermined based on the type of vehicle. The system automatically calculates the toll when a vehicle passes through the toll booth.

Vehicle Type	Toll Amount
Motorcycle	■ 30
Car	■ 60
SUV	■ 80
Bus	■ 120
Truck	■ 150

4.4 File Handling

Theprogram uses C++ file handling to store transaction records in a file called transactions.txt. Each transaction record includes date and time, vehicle plate number, vehicle type, toll booth ID, and toll amount. The system also generates a separate receipt file for each transaction.

5. Implementation Details

5.1 Project File Structure

File	Purpose
main.cpp	Main program containing the complete toll system
transactions.txt	Stores toll transaction history
receipt_<plate>.txt	Stores payment receipt for each vehicle

5.2 Key Implementation Decisions

Menu-DrivenInterface: Thesystem uses a numbered menu that allows users to easily interact with the application and perform different operations such as registering vehicles, processing tolls, and viewing booth statistics.

Static Array for Vehicle Storage: Vehicle records are stored in a fixed-size array within the system. This approach simplifies data management in a console-based application and avoids the overhead of dynamic memory allocation for a simulation context.

Transaction Logging: Every toll payment is recorded in transactions.txt with the date, vehicle details, booth ID, and toll amount. This provides administrators with a complete and persistent history of all transactions.

Receipt Generation: After processing a toll payment, the system generates a dedicated receipt file named using the vehicle's plate number. This makes it easy to retrieve individual transaction records.

5.3 Key Features Implemented

Feature	Description
Vehicle Registration	Stores vehicle plate number, owner name, and type in the system.
Toll Processing	Calculates toll amount automatically based on vehicle type.
Transaction Logging	Saves all payment records to transactions.txt with timestamps.
Receipt Generation	Creates a dedicated receipt text file for each toll payment.
Booth Statistics	Displays total revenue and vehicle count per toll booth.
Menu-driven Interface	Allows full user interaction through a numbered console menu.

6. Results and Conclusion

Findings

TheSmart Toll Collection System successfully demonstrates how toll operations can be automated using a software application. The following key outcomes were observed:

- Vehicle registration works effectively using object-oriented programming with the Vehicle class.
- Toll calculation is automatically performed based on vehicle type with no manual input required.
- Transaction records are stored successfully using file handling in transactions.txt.
- Payment receipts are generated correctly for each transaction.

Limitations

- Thesystem is a simulation and does not connect to real toll booth hardware.
- Vehicle records are stored only during program execution and are not permanently saved across sessions.
- The console interface does not provide graphical visualisation.
- The system does not support online or digital payment systems.

Conclusion

The Smart Toll Collection System demonstrates how software can improve the efficiency of toll management operations. By automating vehicle identification, toll calculation, and transaction recording, the system reduces manual work and improves operational accuracy.

The project highlights the practical application of object-oriented programming and file handling in C++ to develop a structured toll management simulation. The modular class-based design ensures the codebase is maintainable and extensible.

With further development, the system could be expanded to include database integration for persistent vehicle storage, a graphical user interface, digital payment support, and integration with real-time traffic monitoring hardware.

7. References

1. C++ Programming Documentation. https://cplusplus.com

2. Object-Oriented Programming in C++. https://www.geeksforgeeks.org/object-oriented-programming-in-cpp/

3. C++ File Handling Guide. https://www.geeksforgeeks.org/file-handling-c-classes/

4. Electronic Toll Collection Systems. https://en.wikipedia.org/wiki/Electronic_toll_collection

Declaration

I hereby declare that this case study report on Smart Toll Collection System (C++) is my original work. All references and resources used in this report have been properly cited.

Subrata Panda

Roll No: 150096725110

Case Study No: 85

School of Future Tech, ITM Skills University